

TRABAJO PRÁCTICO INTEGRADOR

Ingeniería de Software en la Nube: Creación y Despliegue de Soluciones SaaS Escalables

UTN Facultad Regional La Plata • Departamento de Ingeniería en Sistemas de Información • Ciclo Lectivo 2026

1. VISIÓN ESTRATÉGICA Y OBJETIVOS DEL PROYECTO

El Trabajo Práctico Integrador (TPI) constituye el núcleo pedagógico de esta cursada. No se trata de un simple ejercicio de codificación, sino de un desafío de ingeniería donde el alumno debe demostrar su capacidad para diseñar, implementar y operar soluciones de software bajo estándares de industria. En el contexto de la **Ingeniería en Sistemas de la UTN FRLP**, buscamos formar profesionales que trasciendan la programación local para dominar infraestructuras programables, garantizando que sus productos sean no solo funcionales, sino escalables, resilientes y económicamente sostenibles en entornos productivos.

El objetivo central es la construcción de un **Producto Mínimo Viable (MVP)** bajo el modelo **Software as a Service (SaaS)**, desarrollado en equipos de **3 a 4 integrantes**. El proyecto se fundamenta en los siguientes pilares:

- **Arquitectura Cloud-Native:** Diseño de sistemas pensados desde su concepción para aprovechar la elasticidad, el desacoplamiento y la alta disponibilidad de la nube.
- **Servicios Gestionados:** Uso estratégico de servicios administrados para minimizar la carga operativa y maximizar el *Time-to-Market*.
- **IA como Diferenciador de Negocio:** Integración mandatoria de servicios de Inteligencia Artificial para transformar aplicaciones tradicionales en soluciones inteligentes y competitivas.

En la actualidad, la integración de IA no es un componente opcional o estético; es el motor que redefine la competitividad de un producto. La capacidad de automatizar decisiones, procesar lenguaje natural o realizar análisis predictivos es lo que separa a un sistema convencional de una solución de software moderna. Alcanzar este nivel de sofisticación técnica exige una autonomía profesional absoluta y un criterio de diseño riguroso.

2. MARCO DE LIBERTAD TECNOLÓGICA JUSTIFICADA

La cátedra sostiene una filosofía de **Libertad Tecnológica**. Entendemos que un ingeniero senior no elige herramientas por preferencia personal o moda, sino a través de un proceso de diseño consciente que evalúa trade-offs técnicos y operativos.

"Se permite el uso de cualquier proveedor de nube (AWS, Azure, GCP, Vercel, Supabase) o stack tecnológico, siempre que cada decisión esté técnicamente fundamentada en los requisitos de escalabilidad, costos y arquitectura de la solución propuesta."

Para estructurar la propuesta, se define la siguiente matriz de referencia:

Componente	Tecnología Sugerida / Referencia
Frontend	PaaS (Vercel / NextJS, AWS Amplify)
Backend / API	Serverless (AWS Lambda / FaaS) o Contenedores
Persistencia	SQL Gestionado (RDS) o NoSQL (DynamoDB, MongoDB Atlas)
Storage / CDN	Almacenamiento de objetos y distribución (AWS S3 + CloudFront)
Autenticación	Servicios gestionados (Auth0, Clerk, AWS Cognito)

Esta libertad de elección conlleva la responsabilidad ineludible de documentar y justificar cada componente del stack, asegurando que la arquitectura soporte la carga y disponibilidad exigida para un entorno SaaS real.

3. EL NUEVO ROL DEL INGENIERO: AI DECISION LOG (AI-DECISIONS.MD)

La ingeniería de software ha evolucionado: el profesional ha pasado de ser un escritor de sintaxis a un auditor de arquitecturas y optimizador de soluciones asistidas. En esta cursada, el uso de asistentes (Cursor, GitHub Copilot, Amazon Q) es fomentado, pero bajo un estricto control de calidad humano. El ingeniero es el responsable final del código que llega a producción, independientemente de quién o qué lo haya generado.

Para transparentar este proceso, es obligatorio mantener el archivo `AI-DECISIONS.md` en la raíz del repositorio. Este log de auditoría debe documentar el razonamiento técnico detrás de cada fragmento de código o diseño generado por IA, siguiendo esta estructura:

- **Problema abordado:** Descripción del desafío técnico.
- **Prompt / Herramienta utilizada:** Instrucción enviada y asistente empleado.
- **Código / Arquitectura generada:** Resumen de la propuesta de la IA.
- **Validación y Corrección Humana:** Análisis crítico donde se identifiquen y corrijan alucinaciones, ineficiencias de rendimiento o riesgos de seguridad.

EJEMPLO DE ENTRADA EN EL LOG (AI-DECISIONS.MD)

Refactorización de lógica de persistencia en DynamoDB:

- **Problema:** Optimización de costos en consultas por rango de fecha.
- **Prompt:** "Genera un GSI en TypeScript para filtrar pedidos por createdAt en DynamoDB".
- **AI Output:** Propuso una clave de partición genérica con alta probabilidad de 'hot partition'.
- **Corrección:** Se modificó la clave de partición sugerida por la IA para incluir un prefijo de 'sharding' manual, evitando cuellos de botella detectados en la auditoría técnica.

Este registro será el documento base para evaluar la integridad profesional y el criterio técnico del equipo.

4. EJES TEMÁTICOS E IDEAS DE INSPIRACIÓN

Los siguientes casos son puntos de partida para explorar el potencial de las arquitecturas de nube; se espera que cada equipo proponga una visión superadora:

- **Soporte Inteligente:** Sistema de ticketing SaaS con clasificación automática de incidencias, análisis de sentimiento de los usuarios y priorización dinámica mediante servicios gestionados de procesamiento de lenguaje natural (NLP).
- **RAG (Retrieval-Augmented Generation) de Documentos:** Plataforma de gestión documental corporativa que implementa búsqueda semántica y consultas en lenguaje natural sobre archivos privados, utilizando bases de datos vectoriales y modelos de lenguaje de gran escala (LLM).
- **Auditor Visual:** Herramienta de análisis en tiempo real para la detección de anomalías en procesos industriales o cumplimiento de normativas de seguridad, utilizando servicios gestionados de visión artificial y eventos serverless.

Una visión arquitectónica ambiciosa carece de valor sin la disciplina necesaria para cumplir con el siguiente cronograma de ejecución.

5. CRONOGRAMA DE ENTREGAS E HITOS CRÍTICOS

El proyecto es incremental. La consistencia en el desarrollo es el único camino para asegurar una defensa exitosa y evitar la acumulación de deuda técnica crítica al final del cuatrimestre.

Hito	Fecha	Entregable Principal
Clase 1	24/08/2026	One-Pager: Nombre del proyecto, descripción del problema, propuesta de valor, usuarios objetivo y stack tentativo.
Checkpoint 1	28/09/2026	Definición de Arquitectura: Diagrama Cloud detallado, setup de infraestructura y repositorio inicial con actividad.
Checkpoint 2	09/11/2026	Demo Funcional (Pre-entrega): Sistema con infraestructura desplegada, backend operativo y pruebas de integración.
Defensa Final	30/11/2026	Presentación en vivo: Demostración del MVP en producción y cierre de actas.

El cumplimiento estricto de este calendario es condición necesaria para mantener la regularidad y aspirar a la promoción.

6. ESTÁNDARES DE INGENIERÍA Y EVALUACIÓN CONTINUA

La "Higiene de Ingeniería" define la calidad profesional del equipo. Se evaluará no solo el "qué", sino el "cómo" se construye la solución:

- **Gestión de Código (Git):** Uso obligatorio de *Conventional Commits* (`feat` , `fix` , `docs`). El historial de Git es la evidencia principal para la evaluación individual; se auditará la frecuencia, calidad y autoría de los commits. No se aceptarán "grandes commits" de último momento.
- **Revisiones de Código:** Uso de Pull Requests cruzados entre los miembros del equipo para asegurar la calidad y el conocimiento compartido.
- **Higiene del Kanban:** Gestión de tareas en GitHub Projects con límites de *Work In Progress* (WIP) definidos para evitar cuellos de botella operativos.

- **Observabilidad:** Implementación de métricas, logs centralizados y estrategias de monitoreo para garantizar que el equipo conozca el estado de salud de la solución en tiempo real.
- **Automatización (CI/CD):** Uso de GitHub Actions para validaciones automáticas de sintaxis, pruebas y despliegue continuo.

7. CRITERIOS DE EVALUACIÓN Y CIERRE

La evaluación de esta cátedra es rigurosa y se centra en la capacidad de defensa de las decisiones tomadas. No se evalúa el grupo como una masa uniforme, sino a cada individuo a través de su historial de contribuciones y su solvencia técnica durante la defensa.

La matriz de evaluación prioriza:

- **Entendimiento de la arquitectura:** Justificación técnica de cada servicio cloud utilizado.
- **Calidad del historial en Git:** Trazabilidad de la participación individual.
- **Resiliencia del despliegue:** Funcionamiento real de la solución ante fallas y escalabilidad.
- **Observabilidad y Auditoría:** Correcta gestión de logs y cumplimiento del `AI-DECISIONS.md`.

Como futuros ingenieros, ustedes son los responsables absolutos de la disponibilidad de su solución y de la integridad del código en producción. El éxito depende de su compromiso con la excelencia técnica.